

🔧 Webアプリケーション基本設計書：Retro-Text Board

1. システム概要

本システムは、日本のインターネット文化の象徴である「2ちゃんねる（現5ちゃんねる）」の伝統的なテキストベース掲示板のUI/UXを、モダンなバックエンド技術を用いて再現したスレッド式掲示板アプリケーションです。講義で扱った「NNチャット」の基盤を拡張し、匿名性と堅牢なセキュリティ、そして管理者によるモデレーション機能を備えています。

1.1 主要機能

- **匿名/記名投稿機能:** ユーザー名を任意で入力でき、未入力の場合は自動的に「名無しさん」として処理。
- **ネットワーク情報（ID）の表示:** 投稿者のリモートIPアドレスから一意の識別子（ID）を生成・表示。
- **日時フォーマットの最適化:** 2ちゃんねる標準の表記（年/月/日(曜日) 時:分:秒）への整形。
- **管理者用モデレーション機能:** 管理者（admin）のみが、全投稿に対してサーバーサイドの認可チェックを経た物理削除を実行可能。
- **多層防御セキュリティ:** XSS（クロスサイト・スクリプティング）防御、CSRF（クロスサイト・リクエスト・フォージェリ）対策、および本番環境での強制HTTPS通信。

2. 画面設計 (UI/UX)

Bootstrap 5を用いて、シンプルかつ直感的な「テキスト掲示板」のレイアウトを構築します。

2.1 画面コンポーネント構成（Pugレイアウト）

1. ヘッダー（ナビゲーションバー）：`.navbar.fixed-top`

- タイトル「Retro-Text Board」および現在のログインユーザー名を表示するドロップダウンメニュー（テーマ切り替え、ログアウトリンク内蔵）。

2.

メインコンテンツ（タイムライン）：`.container`（ヘッダーと被らないよう `padding-top: 70px;` を適用）

- 各投稿はシンプルな境界線（`.border-bottom`）で区切られたレス（返信）形式。
- ヘッダー行：`[レス番号]`：`[投稿者名]`：`[投稿日時]`：`[ID: 識別子]`
- 本文行：インデントされたプレーンテキスト（CSSの `white-space: pre-wrap;` で安全な改行を維持）。
- 条件付きコンポーネント：ログインユーザーが「admin」の場合のみ、ヘッダー行に「削除」ボタンを表示。

3. フッター固定書き込みフォーム：`.fixed-bottom`

- 画面下部に常に固定されるグレー背景のフォームエリア。
- 入力項目：「名前（任意・プレースホルダー：名無しさん）」、「本文（`textarea`）」、「書き込む（`button`）」。

3. データベース設計 (Prisma スキーマ)

PostgreSQL上に、投稿データを永続化するための `Post` モデルを定義します。講義のスキーマに、2ちゃんねる特有の「IPアドレス」を記録する `ipAddress` フィールドを追加します。

```
// prisma/schema.prisma

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model Post {
  id          Int      @id @default(autoincrement()) // レス番号のベースとなる一意の
  主キー
  content     String   // 投稿本文
  postedBy    String   @db.VarChar(255)             // 投稿者名（未入力時は"名無しさん"）
  ipAddress   String   @db.VarChar(45)              // 投稿者のリモートIPアドレス（IPv4/IPv6対応）
  createdAt   DateTime @db.Timestamptz(6) @default(now()) // 投稿日時（タイムゾーン付き）
}
```

4. ルーティングおよびAPI設計 (RESTful URI)

HTTPメソッドとURIを適切に使い分け、ステートレスなエンドポイントを定義します。

メソッド	パス	処理概要	遷移先 / レスポンス
GET	<code>/posts</code>	掲示板トップ（全投稿の取得・表示）	
<code>posts.pug</code> レンダリング			
POST <code>/posts</code> 新しいレスの書き込み（DB保存処理）			
<code>/posts</code> へ303リダイレクト（PRGパターン）			

|| **POST** | `/posts/delete` | 管理者による特定のレスの削除（認可チェック必須）

|
`/posts` へ303リダイレクト

|| **GET** | `/logout` | 認証情報のクリアとログアウト画面表示

| ログアウト完了HTML（ログインへのリンク付き）

|| **GET** | `/changeTheme` | ダークモード / ライトモードのCookie反転切り替え

|
`/posts` へ303リダイレクト

5. モジュール設計 (ファイル構成)

責務に応じてコードを「小さな部品（モジュール）」に分割し、可読性と保守性を担保します。

```
├─ index.js           # アプリケーションの起動、HTTPサーバー作成、Basic認証の統合
├─ config.js         # 定数（Cookieキー名など）の一元管理ファイル
├─ package.json      # 依存ライブラリ（http-auth, prisma, dayjs, cookies）の定義
├─ run.sh            # Render起動用のシェルスクリプト（マイグレーション、生成、起動）
├─ lib/
│   ├── router.js     # パス（URI）に基づく各ハンドラーへのルーティング制御
│   ├── posts-handler.js # 掲示板の表示(GET)・投稿(POST)・削除(POST)の専門ロジック
│   └─ handler-util.js # エラーページ(400, 404)、ファビコン、共通HTML等のユーティリティ
└─ views/
    ├── posts.pug      # 掲示板のメインタイムラインテンプレート
    └─ 404.pug         # カスタマイズされた404エラーページ
```

6. セキュリティ・実装ガイドライン

6.1 XSS（クロスサイト・スクリプティング）脆弱性対策

-

サニタイズの徹底: Pugテンプレート内での投稿本文出力には、エスケープを無効化する `!=` は一切使用せず、通常の `=` を用いて安全にテキスト化（無害化）します。

-

多層防御（CSP）: `posts-handler.js` のレスポンスヘッダーに `Content-Security-Policy: default-src 'self';` を設定し、HTML内に悪意あるスクリプトがインラインで挿入されてもブラウザ側で実行を遮断し

ます。

6.2 CSRF（クロスサイト・リクエスト・フォージェリ）対策

-

ワンタイムトークン検証: 新規投稿および削除の `POST` リクエストが発生するたびに、サーバー側で `crypto` モジュールを用いて生成した予測不可能な暗号トークンをフォームに埋め込み、バックエンドで一致検証後に破棄（ワンタイム化）します。

6.3 サーバーサイド認可バリデーション（二重チェック）

-

不正削除のブロック: `/posts/delete` ハンドラー内では、フロントエンドの表示有無に関わらず、`req.user === 'admin'` であるかを必ず検証してから Prisma の `delete` クエリを実行します。認可を満たさないリクエストは `400 Bad Request` として即座に却下します。

6.4 運用環境における多層防御

-

常時HTTPSの強制: `router.js` の冒頭に、本番環境（`process.env.NODE_ENV === 'production'`）かつ非暗号化通信（`req.headers['x-forwarded-proto'] === 'http'`）であるリクエストを検知した場合、即座にアクセスを遮断（または404返却）する防御層を構築します。

この設計書に沿って、まずは `prisma/schema.prisma` のデータベース定義から順に実装を進めていくと、ブレのない堅牢な2ちゃんねる風掲示板が完成します。CLIでの開発、応援しています！